# HKTM CCSDS Signal Generator

## User Manual

CCSDS 131.0-B-5 test signal generation and recorded-pass analysis
for RF-Catcher (TestTree) Capture & Playback


| | |
|---|---|
| Document | HKTM CCSDS Signal Generator – User Manual |
| Software version | 0.1.0 |
| Issue | Issue 1.0 |
| Date | 12 October 2026 |

# Contents

**1 Introduction** **3**

1.1 Purpose . . . 3

1.2 What the tool provides . . . 3

1.3 Reference documents . . . 3

1.4 Abbreviations . . . 3

**2 Installation and start-up** **4**

2.1 Standalone executable . . . 4

2.2 Running from source . . . 5

**3 The signal chain** **5**

3.1 Baseline parameters . . . 5

3.2 Input formats . . . 5

**4 Graphical interface (GUI)** **6**

4.1 Settings sidebar . . . 6

4.2 Signal generator tab: live preview . . . 7

4.3 Signal generator tab: export . . . 8

4.4 Recording analysis tab . . . 9

**5 Command-line tools (CLI)** **11**

5.1 generate_signal.py – generate a signal . . . 11

5.2 analyze_recording.py – analyse a recording . . . 11

5.3 verify_spectrum.py – check an exported file . . . 12

**6 Test scenarios** **12**

6.1 Virtual Channels and Spacecraft ID . . . 12

6.2 Reed-Solomon boundary test . . . 12

6.3 Per-Virtual-Channel error accounting . . . 12

6.4 Transmitter impairments . . . 12

6.5 Doppler of a pass . . . 12

6.6 Reproducing a recorded pass . . . 12

**7 Output files** **13**

**8 Licence administration (issuer)** **13**

**9 Limitations and troubleshooting** **14**

9.1 Known limitations . . . 14

9.2 Troubleshooting . . . 14

**Appendix A CLI option reference** **15**

A.1 generate_signal.py . . . 15

A.2 analyze_recording.py . . . 17

A.3 verify_spectrum.py . . . 17

# 1 Introduction

## 1.1 Purpose

The **HKTM CCSDS Signal Generator** produces baseband IQ test signals that follow the CCSDS telemetry channel-coding chain (CCSDS 131.0-B-5) and writes them in a format that an IQ recorder/replayer can play back at RF – in particular the RF-Catcher (TestTree) Capture & Playback application. The signal can then be injected into a ground-station receiver to test demodulation, decoding, frame synchronisation and Virtual Channel routing with fully known content.

It also analyses real recorded passes (RF-Catcher `.rfcatcher` files or raw IQ): it measures symbol rate, carrier offset, Es/N0 and impairments, decodes the frames, and can copy the measured parameters back into the generator so that a recorded downlink can be reproduced.

## 1.2 What the tool provides

- **Signal generator**: Reed-Solomon, pseudo-randomizer, ASM, convolutional coding with puncturing, NRZ-L, QPSK/BPSK, RRC pulse shaping, all configurable.
- **Test features**: synthetic TM primary headers with selectable Spacecraft ID and Virtual Channels, deterministic Reed-Solomon error injection, transmitter impairments (LO offset, phase noise, IQ imbalance, PA compression) and channel Doppler.
- **Outputs**: raw interleaved IQ (float32 or int16) with a JSON metadata file, or a ready-to-play RF-Catcher `.rfcatcher` recording.
- **Recording analysis**: characterisation and decoding of real recorded passes.
- **Two interfaces**: a graphical interface (GUI) in the web browser with live spectrum and constellation, and command-line tools (CLI) for scripted use. Both use the same signal chain code.

## 1.3 Reference documents

| Ref. | Document |
|---|---|
| [RD-1] | CCSDS 131.0-B-5, TM Synchronization and Channel Coding, Blue Book, September 2023 |
| [RD-2] | CCSDS 132.0-B-3, TM Space Data Link Protocol, Blue Book (primary header only) |
| [RD-3] | ECSS-E-ST-50-01C, Space data links – Telemetry synchronization and channel coding |
| [RD-4] | TestTree RF-Catcher user manual (Capture & Playback application) |
| [RD-5] | PACKAGING.md and README.md in the software repository |

## 1.4 Abbreviations

| Term | Meaning |
|---|---|
| ASM | Attached Sync Marker (0x1ACFFC1D) |
| CADU | Channel Access Data Unit: ASM + Reed-Solomon codeblock(s) |
| Es/N0 | Energy per symbol to noise power spectral density ratio |
| FER | Frame Error Rate |
| IQ | In-phase / Quadrature baseband samples |
| NRZ-L | Non-Return-to-Zero Level line coding |
| PA | Power Amplifier |
| PSD | Power Spectral Density |
| QPSK / BPSK | Quadrature / Binary Phase Shift Keying |
| RRC | Root-Raised-Cosine pulse-shaping filter (roll-off α) |

| Term | Meaning |
|---|---|
| RS | Reed-Solomon code |
| SCID / VCID | Spacecraft Identifier (10 bits) / Virtual Channel Identifier (3 bits) |

# 2 Installation and start-up

The tool can be used in two ways: as a **standalone Windows executable** (no Python needed), or **from the source code** with Python. The functions are the same.

## 2.1 Standalone executable

### What you receive

A zipped folder `HKTM-CCSDS-Signal-Generator` and, separately, a licence file `license.key`.

### Installation

- Unzip the folder anywhere (for example `C:\Tools\`). No installer and no administrator rights are needed.
- Copy `license.key` into that folder, next to `HKTM-CCSDS-Signal-Generator.exe`.
- Keep the `_internal` folder next to the `.exe`: it contains Python and the libraries. The folder is about 500 MB.

```
HKTM-CCSDS-Signal-Generator\
    HKTM-CCSDS-Signal-Generator.exe
    license.key            <- the licence file you received
    _internal\             <- do not move or delete
```

### Starting the program

- Double-click `HKTM-CCSDS-Signal-Generator.exe`.
- A splash screen with the logo shows the start-up progress, and a console window opens.
- After a few seconds the GUI opens in the default web browser at `http://localhost:8501`. If the browser does not open, type that address by hand (the console window also prints it).
- The program runs only on your computer: the browser is just its display. Nothing is sent over the network.
- **To stop the program, close the console window.** Closing the browser tab alone leaves it running.

### Licence

The executable checks the licence file at every start. It holds the customer name and the last valid day, and is signed by the issuer: any change to the file makes it invalid. The sidebar of the GUI shows Licensed to <customer> · valid until <date>; during the last 30 days a warning is shown instead.

| Situation | What happens | What to do |
|---|---|---|
| No `license.key` in the folder | The console prints “No licence found” with the folder to use, and the program stops. | Copy the licence file next to the `.exe`. |
| Licence expired | The console prints the expiry date and the program stops. | Ask the issuer for a new licence file (no new executable is needed). |
| File altered or damaged | “The licence file is not valid” / “damaged”. | Use the original file as received. |
| System date set back | “The system date appears to have been set back”. | Correct the date and time of the computer and restart. |
| Expires within 30 days | The program works; the GUI shows a warning. | Request a renewal. |

Table 1 – Licence messages at start-up

## 2.2 Running from source

Requires Python 3.10 or later. In a terminal, in the repository folder:

```
pip install -r requirements.txt
streamlit run app.py            # GUI, opens in the browser
python generate_signal.py -h    # CLI help
```

When run from source there is no licence check.

# 3 The signal chain

The generator applies the CCSDS channel-coding steps in the order of [RD-1]. Each step can be configured or switched off:

```
payload -> RS(255,223) interleave x5 -> [pseudo-randomizer, ASM excluded]
        -> + ASM (per CADU)  -> convolutional K=7 r=1/2 (over the CADU stream)
        -> NRZ-L -> QPSK (Gray) or BPSK -> RRC -> [impairments] -> [Doppler]
        -> normalisation -> [resampling] -> IQ file
```

## 3.1 Baseline parameters

| Parameter | Baseline value | Options |
|---|---|---|
| Modulation | QPSK (Gray) | QPSK, BPSK |
| Symbol rate | 1 785 000 S/s | any; or set the bit rate (bit rate = symbol rate × 2 for QPSK) |
| Bit rate (channel) | 3 570 000 bps | derived |
| Line coding | NRZ-L (1 → +1, 0 → −1) | NRZ-L only |
| Reed-Solomon | RS(255,223), E = 16, interleave depth I = 5 | on/off; E = 8 or 16; I = 1, 2, 3, 4, 5, 8 |
| Pseudo-randomizer | CLI: none · GUI: long | none; short (255-bit, legacy); long (131071-bit, CCSDS default since 2023) |
| ASM | 0x1ACFFC1D, 4 bytes, not coded by RS | fixed |
| CADU length | 4 + 255 × I = 1279 bytes | depends on I |
| Convolutional | K = 7, rate 1/2, G1 = 171o, G2 = 133o, G2 inverted | on/off; rate 1/2, 2/3, 3/4, 5/6, 7/8 |
| Pulse shaping | RRC, α = 0.35, span 8 symbols, 4 samples/symbol | α, span, samples/symbol |
| Spacecraft ID | 0x123 (291) | 0–1023; written only with Virtual Channels enabled |
| Output | float32 IQ, peak 0.9, native rate (7.14 MS/s) | int16, .rfcatcher, peak amplitude, resampling to a fixed rate |

Table 2 – Baseline parameters

## 3.2 Input formats

The payload can be pseudo-random (reproducible with a seed) or read from a file. The input format says what the data already contains, so that no step is applied twice:

| Input format | File content | Steps applied by the generator |
|---|---|---|
| Transfer Frame (default) | Raw, uncoded Transfer Frames | RS, randomizer, ASM, convolutional, modulation |
| ASM + Transfer Frame | ASM + frame, not scrambled (e.g. decoded output of a ground station, or frames saved by the recording analysis) | Randomizer (never on the ASM; default short, “none” not allowed), convolutional, modulation. No RS, no new ASM. |
| CADU as on air | ASM + already RS-coded and scrambled block | Convolutional and modulation only (randomizer must be “none”) |

Table 3 – Input formats

With file input, each record is exactly 4 + 255 × I bytes. The tool searches for the first real ASM, so a file need not start on a CADU boundary, and skips any recorder-added bytes between the end of one CADU and the next ASM.

# 4 Graphical interface (GUI)

The GUI has a sidebar with the generator settings and two tabs: **Signal generator** and **Recording analysis**. Every change of a setting recomputes a short preview of the signal immediately.

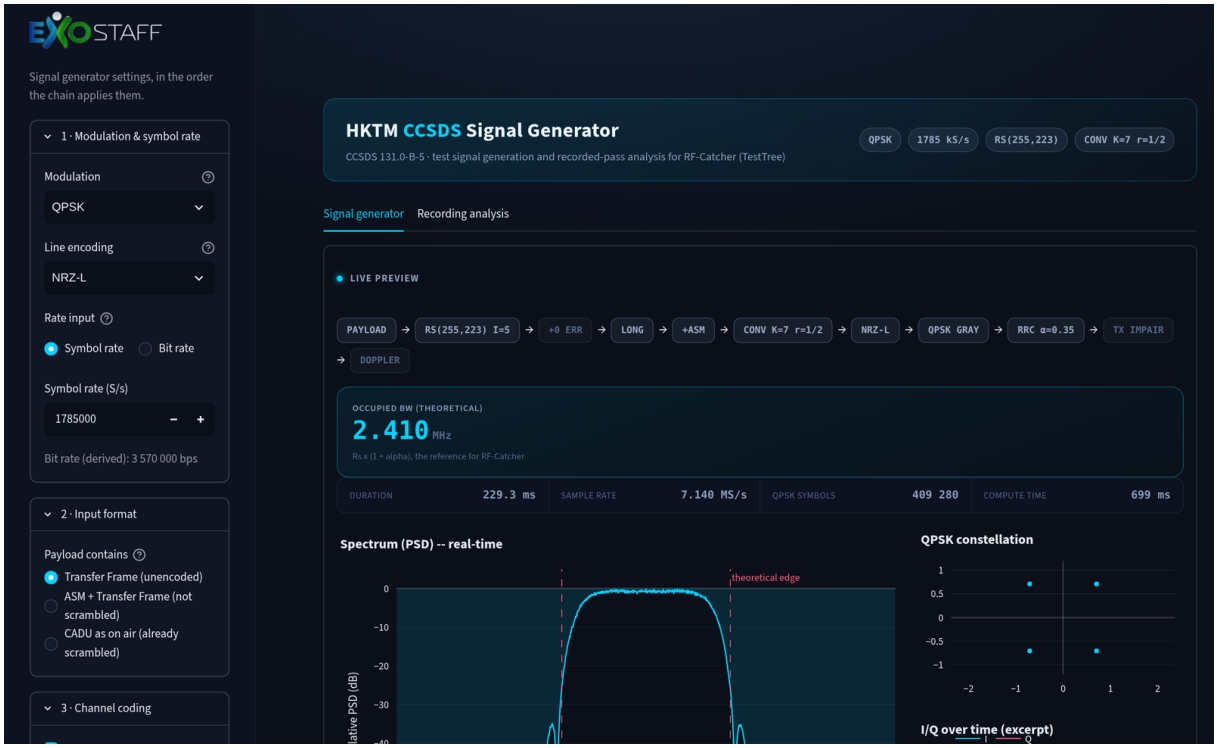



Figure 1 – Main window: settings sidebar, header with the active baseline, the two tabs and the live preview of the signal generator

## 4.1 Settings sidebar

The settings are grouped in the order the chain applies them. Each field has a help icon (?) with a short explanation.

| Section | Content |
|---|---|
| 1 · Modulation & symbol rate | QPSK/BPSK, line coding, symbol rate or bit rate (the other is derived). |
| 2 · Input format | What the payload contains (Table 3). |
| 3 · Channel coding | Reed-Solomon on/off, E, interleave depth; convolutional on/off, code rate, G2 inversion; pseudo-randomizer. |
| 4 · Payload & duration | Pseudo-random (seed) or file; preview CADU count; **Assign Virtual Channels** (VCID list) and **Spacecraft ID**. |
| 5 · Deterministic error injection | Exact number of RS symbol errors in a chosen codeword of chosen CADUs or of one Virtual Channel. |
| 6 · Pulse shaping (RRC) | Roll-off α and filter span. |
| 7 · Impairments & Doppler | LO frequency offset and phase noise; IQ gain and phase imbalance; PA saturation and AM-PM; channel Doppler shift and rate. |

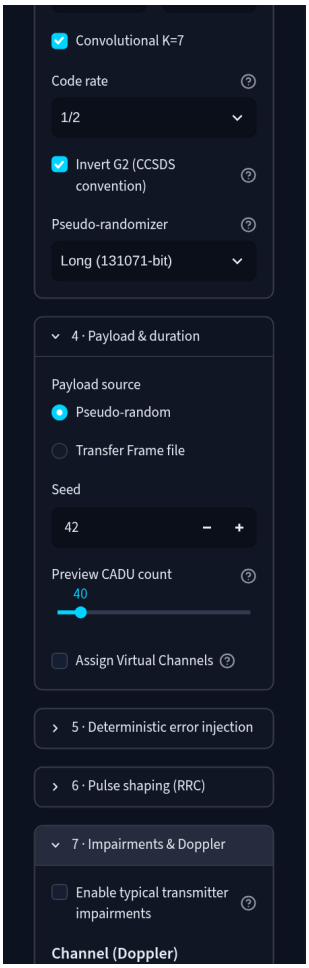



Figure 2 – Sidebar sections (right: part of the sidebar with section 7 open)

## 4.2 Signal generator tab: live preview

The **Live preview** panel shows:

- the chain stages as a row of labels: active stages are highlighted, disabled ones are greyed out;
- the theoretical occupied bandwidth Rs × (1 + α), duration, sample rate, number of symbols and compute time of the preview;
- the **spectrum** (PSD) with the theoretical band edges as dashed red lines;
- the **constellation**, taken after a matched filter, so impairments (phase noise, IQ imbalance, PA) are visible;
- an **I/Q excerpt** over time.

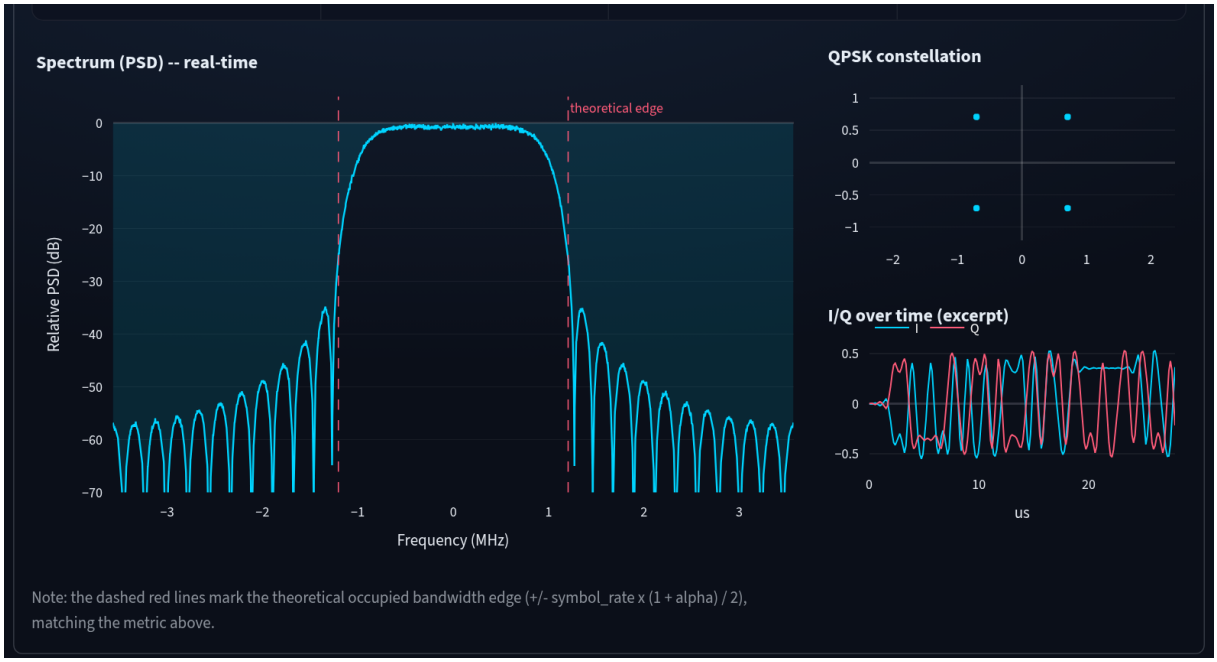



Figure 3 – Live preview: spectrum with theoretical band edges, constellation and I/Q excerpt

The preview uses only a few tens of CADUs to stay fast. The length of the exported file is set separately in the Export panel.

## 4.3 Signal generator tab: export

The **Export** panel writes the full-length file:

| Field | Meaning |
|---|---|
| Export CADU count | Length of the file. The panel shows the resulting duration and file size. |
| Output format | **RF-Catcher recording (.rfcatcher)**: carries its own sample rate, RF frequency and bandwidth. **Raw IQ int16** (RF-Catcher sample format) or **raw IQ float32**: no header; rate and frequency must be set by hand on the replayer. |
| Peak amplitude | Normalised peak (default 0.9), headroom against clipping. |
| Resample to fixed rate | Resample to an exact rate accepted by the replayer, e.g. 10 000 000 Hz. Without it, the native rate is symbol rate × 4. |
| RF center frequency | Written into the .rfcatcher metadata (default 1707 MHz). |
| Bandwidth: automatic | RF-Catcher's linked value, sample rate / 1.1; untick to enter it. |
| Device metadata from | Optional real .rfcatcher recording whose device fields (serial number, firmware, gain) are copied. |
| Generate export file | Creates the file in the `output` folder and offers the download buttons (file and metadata). Files above 1 GB stay in the folder only. |

Table 4 – Export panel

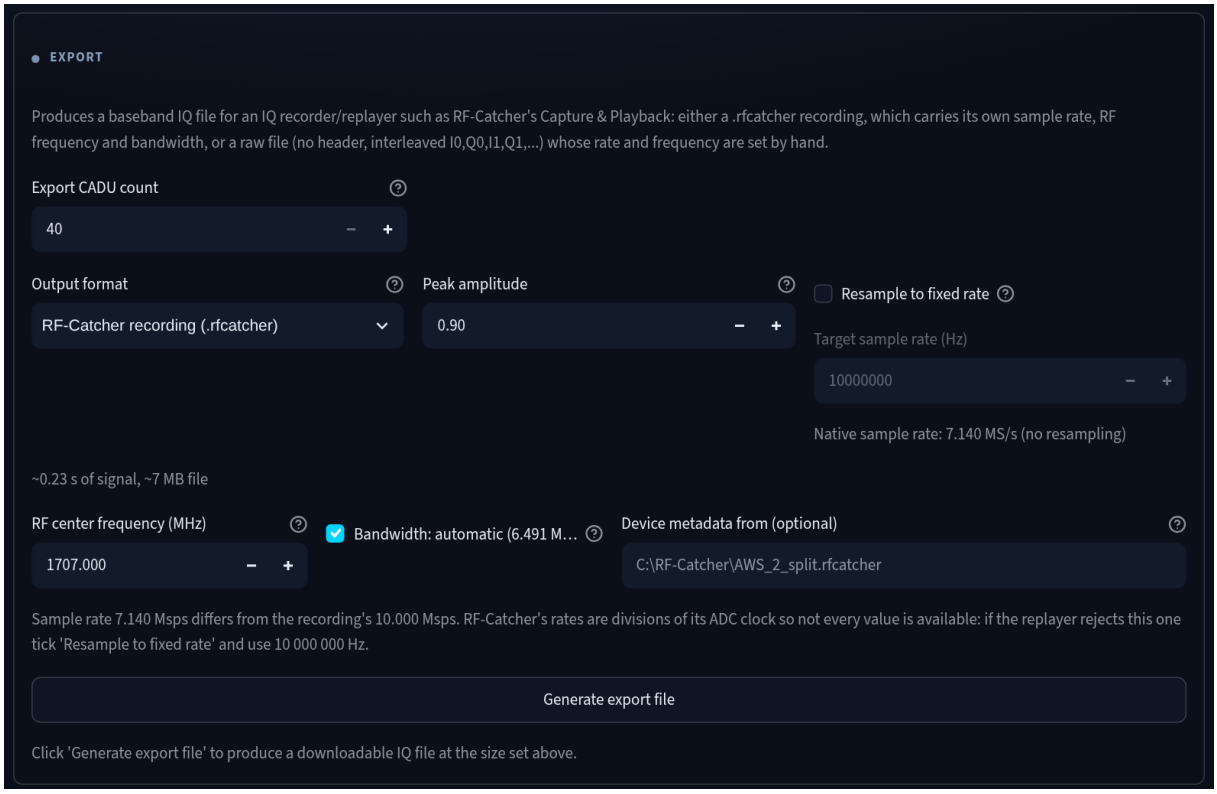



Figure 4 – Export panel with the .rfcatcher output format

RF-Catcher only accepts some sample rates (divisions of its ADC clock). If the replayer refuses the native rate, tick **Resample to fixed rate** and use 10 000 000 Hz. The panel warns when the rate differs from the template recording.

## 4.4 Recording analysis tab

This tab characterises a real recorded pass and can transfer the result to the generator.

- Enter the **local path** of the recording (.rfcatcher or raw IQ). The file is read directly from disk, not uploaded, so multi-GB files are fine.
- Choose the slice: **Start at** and **Duration** (up to 2 s; 1 s is plenty). Only that slice is read.
- For raw IQ, give the sample rate and sample format; for .rfcatcher they come from the metadata.
- Nominal symbol rate and matched-filter roll-off are starting values for the estimation.
- **Decode frames** runs the Viterbi decoder and the frame analysis (slower: about 15 s).
- Click **Analyze recording**.

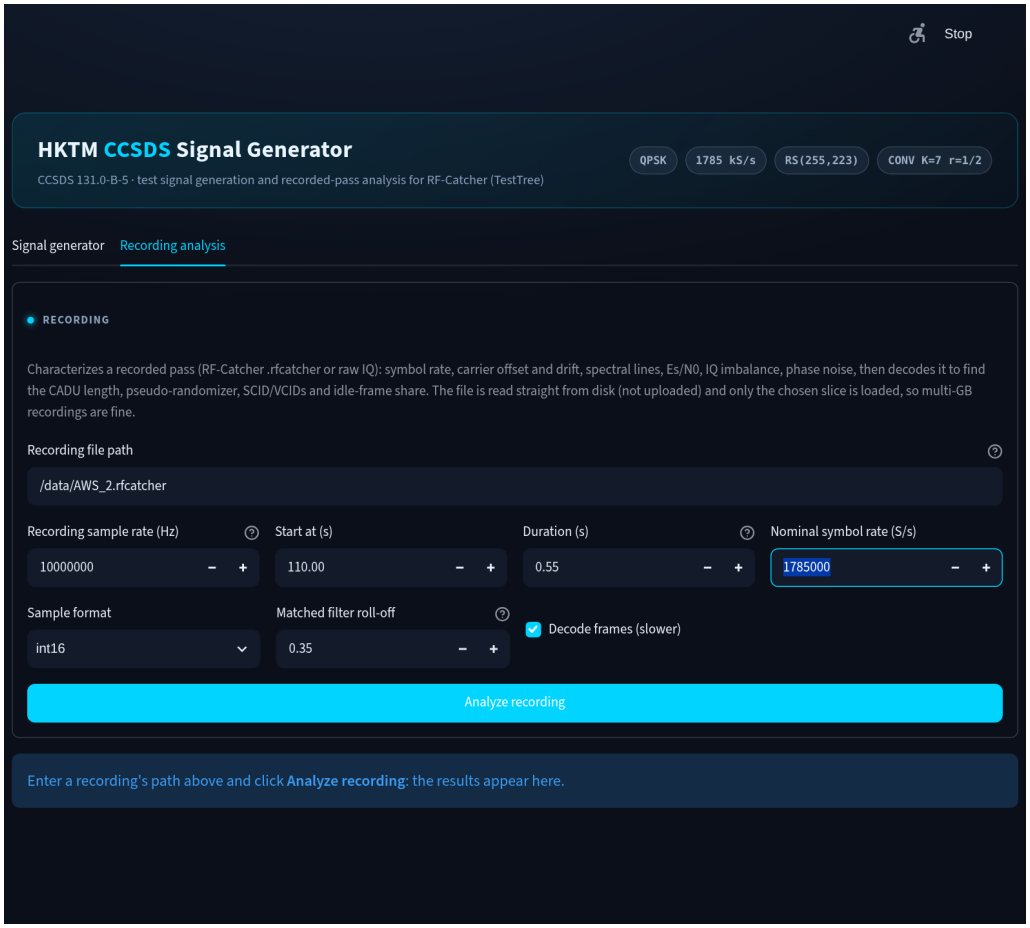



Figure 5 – Recording analysis: input fields

The results show symbol rate (and offset in ppm), carrier offset and drift, Es/N0 and EVM, the number of discrete spectral lines, the randomizer and CADU length found, the share of idle frames and the Spacecraft ID; then the spectrum, the constellation after carrier recovery and the residual carrier phase. **Full report** lists all values, including IQ imbalance, phase noise and the Virtual Channels found.




Figure 6 – Recording analysis: results and the “Reproduce it in the generator” panel

### Reproduce it in the generator

Tick the values to copy (symbol rate, rounded to 50 Hz; modulation; randomizer; optionally the carrier offset) and click **Apply to generator**: the generator tab opens with those settings. **Download the decoded frames** saves them as ASM + Transfer Frame records; load that file as payload with input format “ASM + Transfer Frame” to regenerate the recorded content bit for bit.

# 5 Command-line tools (CLI)

Three scripts cover the same functions as the GUI. All options are listed in Appendix A and by `-h`.

## 5.1 generate_signal.py – generate a signal

```
python generate_signal.py                       # baseline, file in output/
python generate_signal.py --n-cadu 500 --randomizer long -o test.raw
python generate_signal.py --rs-e 8 --interleave-depth 2 --conv-rate 3/4 -o test2.raw
python generate_signal.py --modulation BPSK -o test3.raw
python generate_signal.py --dtype int16 --target-fs 10e6 --peak 0.9
python generate_signal.py --rfcatcher --target-fs 10e6 --rf-frequency-mhz 1707
```

Without `-o`, the file is written to `output/qpsk_ccsds_<fs>Msps_<n_cadu>cadu_<timestamp>.iq`. Generation runs in batches, so memory use does not grow with the file length (resampling is the only exception: very long files with resampling are refused before starting).

## 5.2 analyze_recording.py – analyse a recording

```
python analyze_recording.py C:\RF-Catcher\AWS_2_split.rfcatcher --offset 30 --duration 1 --plot aws2.png
python analyze_recording.py capture.iq --fs 10e6 --dtype int16 --save-frames frames.bin
```

It prints the same report as the GUI. `--plot` saves spectrum, constellation and residual phase as PNG; `--save-frames` writes the decoded frames in the “ASM + Transfer Frame” input format. About 15 s and 2 GB of memory per second of signal, plus about 15 s of decoding (limited by default to the first 400 000 symbols, about 40 CADUs).

### 5.3 verify_spectrum.py – check an exported file

```
python verify_spectrum.py output/qpsk_ccsds_....iq --plot spectrum.png
```

Computes the PSD of a generated file (reading its .meta.json for rate and format) and measures the −3 dB and null-to-null bandwidths. For the RRC baseline (Rs = 1.785 MS/s, α = 0.35): −3 dB ≈ Rs, null-to-null ≈ Rs (1 + α) ≈ 2.41 MHz.

## 6 Test scenarios

Typical uses, with the CLI command. The same settings are available in the GUI sidebar.

### 6.1 Virtual Channels and Spacecraft ID

Builds a real 6-byte TM primary header [RD-2] into each frame: Spacecraft ID, Virtual Channel ID (round-robin over the list), master and virtual channel frame counters. Repeating a VCID gives it a larger share. Use it to check that a receiver identifies and routes frames per Virtual Channel.

```
python generate_signal.py --vcid-list 0,0,1,2 --spacecraft-id 291 -o vc_test.raw
```

The Spacecraft ID (10 bits, 0–1023, default 0x123 = 291) is written only together with Virtual Channels (`--vcid-list` / “Assign Virtual Channels”). It cannot be combined with a payload file, because it would overwrite the first 6 bytes of real data.

### 6.2 Reed-Solomon boundary test

Flips an exact number of RS symbols in one codeword of chosen CADUs. With E = 16, 16 errors must be corrected; 17 must be detected as uncorrectable and never miscorrected.

```
python generate_signal.py --corrupt-rs-symbols 16 --corrupt-cadu-indices 10 -o rs16.raw
python generate_signal.py --corrupt-rs-symbols 17 --corrupt-cadu-indices 10 -o rs17.raw
```

### 6.3 Per-Virtual-Channel error accounting

```
python generate_signal.py --vcid-list 0,1,2 --corrupt-rs-symbols 17 --corrupt-vc 1 -o vc1_err.raw
```

Every VC 1 frame is uncorrectable; the receiver's FER report must show errors on VC 1 only.

### 6.4 Transmitter impairments

```
python generate_signal.py --freq-offset-hz 5000 --iq-gain-imbalance-db 0.8 \
    --iq-phase-imbalance-deg 3 --phase-noise-linewidth-hz 200 -o tx_impair.raw
python generate_signal.py --pa-backoff-db -6 --pa-smoothness 3 --pa-am-pm-deg-per-db 3 -o pa_sat.raw
```

Checks carrier recovery, image rejection and tolerance to EVM. A PA driven into saturation produces spectral regrowth outside the occupied band.

### 6.5 Doppler of a pass

```
python generate_signal.py --doppler-hz 30000 --doppler-rate-hz-s -400 -o doppler.raw
```

Frequency shift at the start of the file plus a constant rate (Hz/s): a linear approximation of the Doppler curve of a LEO pass, to check the receiver's carrier tracking.

### 6.6 Reproducing a recorded pass

- Analyse 1 s of the recording (GUI Recording analysis tab, or `analyze_recording.py --save-frames frames.bin`).
- Apply symbol rate, modulation and randomizer to the generator.
- Load the decoded frames as payload with input format “ASM + Transfer Frame”.
- Export as .rfcatcher at the recorder's sample rate and replay it.

# 7 Output files

| File | Content |
|---|---|
| `*.iq` / `*.raw` (float32) | Interleaved I0, Q0, I1, Q1, … as 32-bit floats in [−1, +1]. No header. |
| `*.iq` / `*.raw` (int16) | Interleaved 16-bit little-endian integers, 12 significant bits, range [−2048, 2047] (RF-Catcher format). No header. |
| `*.meta.json` | All parameters used, actual sample rate and format of the file. Note: a raw file is baseband; the RF frequency must be set on the replayer. |
| `*.rfcatcher` | RF-Catcher recording: a tar archive of `<name>.iq` (int16) and `<name>.json` (sample rate, RF frequency, bandwidth, duration, device fields). Headers are written exactly as RF-Catcher writes them. |

Table 5 – Output files

All files are written to the `output` folder unless another path is given.

# 8 Licence administration (issuer)

This chapter is for the person who distributes the executable. Users only need section 2.1. The tool `license_tool.py` is run from the source code and is not part of the executable.

| Step | Command | Notes |
|---|---|---|
| 1. Once: key pair | `python license_tool.py keygen --private-key C:\secure\hktm_license_private_key.pem` | The private key stays with the issuer, outside the repository. The public key is written to assets/license_public_key.pem and is built into every executable made from then on. |
| 2. Build | `pyinstaller hktm_simulator.spec` | See PACKAGING.md. Builds without the public key have no licence check. |
| 3. Per customer | `python license_tool.py issue --private-key C:\secure\hktm_license_private_key.pem --customer "Name" --expires 2027-12-31 -o license.key` | The expiry date is the last valid day. Send license.key with the zipped folder. |
| 4. Renewal | `Same as step 3 with a new date` | No new executable needed. |
| Check | `python license_tool.py verify license.key` | Prints VALID or the reason. |

Table 6 – Licence administration

The licence is a deterrent, not copy protection. Never commit the private key or issued licence files; both are excluded by .gitignore. If the private key is lost, a new key pair, new licences and a rebuild are needed.

# 9 Limitations and troubleshooting

## 9.1 Known limitations

- Turbo and LDPC codes (CCSDS 131.0-B-5 sections 6–8) are not implemented.
- Transfer Frame length constraints and RS virtual fill are not implemented: with QPSK, some combinations of punctured code rate and CADU count give an odd number of coded bits and are refused with an error. The baseline (rate 1/2) is never affected.
- Only NRZ-L with polarity 1 → +1; check the polarity expected by the receiver.
- Only the TM primary header is synthesised: no secondary header, no Space Packets, no frame CRC.
- Pulse shaping is always RRC: an unshaped (NRZ, rectangular pulse) signal is not produced.
- The .rfcatcher output matches a real recording byte for byte, but playback on the instrument has not been confirmed yet.
- The RRC filter adds a transient of half the filter span at the start and end of the file.

## 9.2 Troubleshooting

| Symptom | Cause / action |
|---|---|
| The executable starts slowly the first time | Windows scans the new files; later starts take a few seconds. Wait for the splash screen to close. |
| Browser does not open | Open http://localhost:8501 by hand. If the port is in use, close other instances of the program. |
| Program stops at start-up with a licence message | See Table 1 (section 2.1). |
| “odd number of coded bits” error | Change the CADU count or use convolutional rate 1/2 (see limitations). |
| Error about CADU length with a payload file | RS E and interleave depth do not match how the file's CADUs were built; set them to the real values. |
| Replayer refuses the sample rate | Export with Resample to fixed rate, e.g. 10 000 000 Hz. |
| Recording analysis finds no ASM | Wrong modulation or symbol rate, too low Es/N0, or a slice without signal; try another slice or switch off decoding to see the measurements. |
| An error in the console of the executable | Report the full text of the console window to the supplier. |

Table 7 – Troubleshooting

# Appendix A CLI option reference

Generated from the scripts' own argument parsers when this manual was built.

## A.1 generate_signal.py

| Option | Default | Description |
|---|---|---|
| `--n-cadu N_CADU` | `100` | See section 5. |
| `--modulation {QPSK,BPSK}` | `QPSK` | QPSK (baseline, 2 bits/symbol) or BPSK (1 bit/symbol, half the bit rate at the same symbol rate -- the modulation the base rate-1/2 convolutional code's G2 inversion, CCSDS 3.3.1(5), is specified against) |
| `--alpha ALPHA` | `0.35` | See section 5. |
| `--sps SPS` | `4` | See section 5. |
| `--span SPAN` | `8` | See section 5. |
| `--no-rs` | `off` | disable Reed-Solomon FEC |
| `--rs-e {8,16}` | `16` | RS error correction capability |
| `--interleave-depth {1,2,3,4,5,8}` | `5` | See section 5. |
| `--no-conv` | `off` | disable convolutional FEC |
| `--conv-rate {1/2,2/3,3/4,5/6,7/8}` | `1/2` | See section 5. |
| `--no-invert-g2` | `off` | See section 5. |
| `--randomizer {none,short,long}` | — | CCSDS pseudo-randomizer: 'long' (131071-bit, current standard default), 'short' (255-bit, legacy), or 'none'. Default: 'short' with --input-format asm_frame (where 'none' is rejected), otherwise 'none'. Must be 'none' with --input-format cadu |
| `--input-format {transfer_frame,asm_frame,cadu}` | `transfer_frame` | what --payload-source/the generated payload represents: 'transfer_frame' (raw, uncoded data -- RS/randomizer/ASM applied here, default), 'asm_frame' (ASM + Transfer Frame, NOT scrambled -- scrambled here, never the ASM) or 'cadu' (CADU exactly as on the air, already scrambled -- used verbatim). Neither ASM-framed format gets RS or a new ASM; each record is 4 + 255*I bytes, extra bytes before the next ASM are skipped. The convolutional stage still runs on all formats |
| `--payload-source PAYLOAD_SOURCE` | — | See section 5. |
| `--seed SEED` | `42` | See section 5. |
| `--vcid-list VCID_LIST` | — | comma-separated Virtual Channel IDs (0-7), e.g. '0,1,2', round-robined across generated Transfer Frames: builds a real CCSDS TM primary header (132.0-B-3 4.1.2) carrying each frame's assigned VCID, for validating a receiver correctly identifies/routes frames by Virtual Channel. Repeat a value to weight it more heavily (e.g. '0,0,1' gives VC 0 twice VC 1's share). Only valid with --input-format transfer_frame and no --payload-source (it would overwrite the first 6 bytes of real data); default: no synthetic header, unchanged payload |
| `--spacecraft-id SPACECRAFT_ID` | `291` | 10-bit Spacecraft ID (0-1023) for the synthetic primary header when --vcid-list is set; otherwise unused |
| `--corrupt-rs-symbols CORRUPT_RS_SYMBOLS` | `0` | deterministically flip exactly this many RS symbols within one interleaved codeword (--corrupt-codeword-index) of each targeted CADU (--corrupt-cadu-indices and/or --corrupt-vc), to boundary-test a receiver's RS decoder against its declared correction capability E (--rs-e): E errors must still decode perfectly, E+1 must fail -- an exact, reproducible per-codeword error count a replayer's AWGN can't guarantee hitting. Needs --no-rs NOT set (or --input-format cadu, already RS-coded); default: 0 (disabled) |

| Option | Default | Description |
|---|---|---|
| `--corrupt-codeword-index CORRUPT_CODEWORD_INDEX` | `0` | which of the --interleave-depth interleaved RS codewords to target (0-based); only used with --corrupt-rs-symbols |
| `--corrupt-cadu-indices CORRUPT_CADU_INDICES` | — | comma-separated 0-based CADU indices to corrupt, e.g. '0,5,10'; combines with --corrupt-vc (either match corrupts that CADU) -- at least one of the two is required with --corrupt-rs-symbols |
| `--corrupt-vc CORRUPT_VC` | — | corrupt every CADU assigned to this Virtual Channel ID (requires --vcid-list) -- e.g. to verify a receiver's per-VC FER accounting attributes errors to the right channel only |
| `--corrupt-seed CORRUPT_SEED` | `777` | seed for which symbol positions/values get flipped -- deterministic and reproducible across runs |
| `--freq-offset-hz FREQ_OFFSET_HZ` | `0.0` | residual LO frequency offset applied to the pulse-shaped IQ, in Hz (positive or negative) -- for validating a receiver's carrier-recovery loop against a real (imperfect) transmitter LO, not just a perfectly on-frequency signal; default: 0 (disabled) |
| `--iq-gain-imbalance-db IQ_GAIN_IMBALANCE_DB` | `0.0` | IQ modulator I/Q branch gain mismatch, in dB -- produces a mirror-image tone in the spectrum; default: 0 (disabled) |
| `--iq-phase-imbalance-deg IQ_PHASE_IMBALANCE_DEG` | `0.0` | IQ modulator phase deviation from ideal 90-degree I/Q separation, in degrees -- combines with --iq-gain-imbalance-db in the same mirror-image model; default: 0 (disabled) |
| `--phase-noise-linewidth-hz PHASE_NOISE_LINEWIDTH_HZ` | `0.0` | free-running-oscillator single-sideband 3 dB linewidth, in Hz -- generates Wiener (random-walk) phase noise on the IQ, for validating a receiver's tolerance to constellation smearing from a real transmitter LO; default: 0 (disabled) |
| `--impairment-seed IMPAIRMENT_SEED` | `2718` | seed for the phase noise random walk -- deterministic and reproducible across runs |
| `--doppler-hz DOPPLER_HZ` | `0.0` | channel Doppler shift, in Hz, at the start of the exported signal (not a transmitter impairment -- applied last, after the transmitter chain, since it's imparted by relative spacecraft/ground-station motion along the propagation path); positive or negative; default: 0 (disabled) |
| `--doppler-rate-hz-s DOPPLER_RATE_HZ_S` | `0.0` | constant Doppler rate of change, in Hz/s -- combined with --doppler-hz as a local-linear (chirp) approximation of a real LEO pass's Doppler curve, for validating a receiver's carrier-tracking loop against realistic pass dynamics, not just a static frequency error; default: 0 (disabled) |
| `--pa-backoff-db PA_BACKOFF_DB` | — | power amplifier saturation point (Rapp AM-AM model), in dB above this project's reference unit amplitude (1.0, an ideal symbol's own magnitude) -- models PA compression/saturation, the last physical stage before the antenna: negative values saturate more aggressively (visible spectral regrowth just outside the occupied bandwidth); default: not set (disabled) |
| `--pa-smoothness PA_SMOOTHNESS` | `3.0` | Rapp model knee sharpness (higher = sharper transition from linear to saturated); only used with --pa-backoff-db |
| `--pa-am-pm-deg-per-db PA_AM_PM_DEG_PER_DB` | `0.0` | AM-PM conversion, in degrees of phase shift per dB of AM-AM compression (the same figure real TWTA/SSPA datasheets quote); 0 (default) disables it; only used with --pa-backoff-db |
| `--dtype {float32,int16}` | `float32` | IQ sample format for the output file; int16 is the RF-Catcher format (little-endian, 12 significant bits, range [-2048, 2047]) |
| `--peak PEAK` | `0.9` | normalized peak amplitude before dtype conversion (headroom against clipping) |
| `--target-fs TARGET_FS` | — | resample the output to this exact sample rate in Hz (e.g. 10e6); default: no resampling, native symbol_rate*sps |
| `--rfcatcher` | `off` | write an RF-Catcher recording (.rfcatcher: int16 IQ + JSON metadata with sample rate, RF frequency and bandwidth) instead of a raw IQ file; implies --dtype int16 |
| `--rf-frequency-mhz RF_FREQUENCY_MHZ` | `1707.0` | RF center frequency written to the .rfcatcher metadata (default 1707 MHz) |

| Option | Default | Description |
|---|---|---|
| `--rf-bandwidth-mhz RF_BANDWIDTH_MHZ` | – | bandwidth written to the .rfcatcher metadata (default: RF-Catcher's linked value, sample rate / 1.1) |
| `--rfcatcher-template RFCATCHER_TEMPLATE` | – | a real .rfcatcher recording (or its .json) whose device fields (serial, firmware, gain, level) are copied into the new file's metadata; default: built-in AWS_2 ones |
| `-o, --output OUTPUT` | – | output path; default: output/qpsk_ccsds_<fs>Msps_<n_cadu>cadu_<timestamp>.iq |

## A.2 analyze_recording.py

| Option | Default | Description |
|---|---|---|
| `recording` | `(required)` | .rfcatcher (tar) or raw interleaved IQ file |
| `--fs FS` | – | recording sample rate, Hz (e.g. 10e6); default: read from the .rfcatcher metadata |
| `--dtype {int16,float32}` | `int16` | See section 5. |
| `--header-bytes HEADER_BYTES` | `0` | bytes to skip in a raw (non-tar) file |
| `--offset OFFSET` | `0.0` | start of the analyzed slice, s |
| `--duration DURATION` | `1.0` | length of the analyzed slice, s |
| `--rs-nominal RS_NOMINAL` | `1785000.0` | expected symbol rate, sps |
| `--alpha ALPHA` | `0.35` | RRC roll-off of the matched filter |
| `--max-carrier-offset MAX_CARRIER_OFFSET` | `200000.0` | carrier search range, +-Hz |
| `--modulation {auto,QPSK,BPSK}` | `auto` | See section 5. |
| `--decode-symbols DECODE_SYMBOLS` | `400000` | max symbols to Viterbi-decode (default ~40 CADUs, ~15 s; roughly 20-30 s per million) |
| `--no-decode` | `off` | See section 5. |
| `--plot PLOT` | – | save a spectrum/constellation/phase PNG here |
| `--save-frames SAVE_FRAMES` | – | write the decoded frames here as ASM + de-randomized Transfer Frame records (the generator's 'asm_frame' input format) |

## A.3 verify_spectrum.py

| Option | Default | Description |
|---|---|---|
| `iq_file` | `(required)` | See section 5. |
| `--sample-rate SAMPLE_RATE` | – | override meta.json output_sample_rate (Hz) |
| `--dtype {float32,int16}` | – | override meta.json output_dtype |
| `--nperseg NPERSEG` | `4096` | See section 5. |
| `--plot PLOT` | – | save spectrum plot PNG to this path |